

## Contents

|                                                                                                                                                      |          |
|------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| <b>The Experience Substrate: Frozen Weights, Learned Worlds — Geometric Writeback and World-State Weight Residency for Runtime Agent Populations</b> | <b>1</b> |
| Abstract                                                                                                                                             | 2        |
| 1. The requirement no frozen checkpoint meets                                                                                                        | 2        |
| 2. Two artifacts that turn out to be one                                                                                                             | 4        |
| 3. Architecture: competence and experience                                                                                                           | 4        |
| 4. The format ambiguity, and its resolution                                                                                                          | 6        |
| 5. Execution model: retrieval inside the decode loop                                                                                                 | 7        |
| 6. World-state weight residency                                                                                                                      | 9        |
| 7. One math, three targets                                                                                                                           | 10       |
| 8. What is measured                                                                                                                                  | 11       |
| 9. Corrections ledger: auditing the first implementation                                                                                             | 13       |
| 10. Positioning                                                                                                                                      | 15       |
| 11. Limitations and honest framing                                                                                                                   | 15       |
| 12. Reproducibility                                                                                                                                  | 16       |
| 13. Conclusion                                                                                                                                       | 16       |
| Related work in this series                                                                                                                          | 17       |

## The Experience Substrate: Frozen Weights, Learned Worlds — Geometric Writeback and World-State Weight Residency for Runtime Agent Populations

---

**Authors:** Weslyn Cory Whitehead Jr.<sup>1</sup>

**Affiliations:** <sup>1</sup> AsAManThinks Research, Berkeley, CA, USA

**Corresponding author:** weslyn@asamanthinks.com

**ORCID:** <https://orcid.org/0009-0005-7707-3210>

**Submitted:** August 2026 **Working Paper Series:** AAMT-WP-26 **Anchor DOI:** 10.5281/zenodo.19600795 **License:** Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International (CC BY-NC-ND 4.0). Commercial use, derivative works, and training or evaluation of machine-learning systems on this text require a separate written licence — see LICENSE-COMMERCIAL.md or contact weslyn@asamanthinks.com.

**Status:** Working paper, not peer reviewed. The reference implementation is the MaiaM Unity platform (Assets/MaiaM/Systems/Vortex/, Assets/MaiaM/Systems/Substrate/, Assets/MaiaM/Harmonic/) running against packages/libaamt and the WebGPU engine in packages/voxel-runtime. Section 8 states precisely which quantities are measured, which are derived from measured inputs, and which the harness exists to produce but has not yet produced — these are marked as pending rather than estimated. Section 9 is a corrections ledger against the first Unity implementation of this architecture; unusually for this series, the falsifying evidence is a code audit rather than a benchmark, and the ledger says so. This paper composes **WP-20** (voxel-addressable memory), **WP-21** (voxel-steered generation), **WP-22** (Odu coverage maps) and **WP-23** (the .aamt substrate) into a single runtime claim that none of them makes alone.

---

## Abstract

A generative world populated by language-model agents faces a requirement that no deployment of a frozen checkpoint satisfies: the world must get *wiser* — its agents must carry forward what they learned from what went wrong — while the model that animates them stays fixed, auditable, and small enough to ship. Fine-tuning cannot serve this. It is offline, it is expensive, it destroys the provenance of any individual lesson, and it makes every behavioral change irreversible and unattributable.

We present the **experience substrate**: an architecture in which a frozen, quantized model supplies *competence* and an append-only, geometrically addressed store supplies *experience*, joined by the fact that both are indexed in the same 4-dimensional TERA coordinate space. At decode time the agent’s hidden state is projected to a TERA coordinate (WP-15/WP-20), routed through the substrate’s hexadeca-tree in  $O(\text{depth})$  (WP-23), and the retrieved records bias the logits (WP-21). At deed time the agent’s experience is admitted to the substrate only if it passes the HeartScale/Ma’at evenly-yoked gate, and is recorded in the substrate’s lineage journal, so every behavioral change in the world is attributable to a specific admitted deed and is deterministically replayable.

We then observe that the same coordinate that addresses experience can address *weights*. Quantized weight pages laid out in Morton order and metered by the breath scheduler make VRAM residency a function of where in the world the agent is standing: a district that exercises seven of sixteen Meji orthants need not hold the pages serving the other nine. This reframes a model too large to ship as a model whose working set is bounded by world state rather than by parameter count.

Finally we report an implementation audit. The first Unity implementation of this architecture appeared complete and was inert: its compute kernels never loaded, its uniforms never bound, and its substrate service was pointed at a model container it could not parse and silently fell back from. Section 9 records each failure, why none of them raised an error, and the specific structural changes — not the specific patches — that make each one impossible to reintroduce. That section is the most useful part of this paper for anyone building the same thing.

---

## 1. The requirement no frozen checkpoint meets

The MaiaM world builder states its own requirement plainly: non-player agents construct the world they inhabit at runtime, and *feed knowledge and wisdom learned from mistakes back into the system* so that the system becomes better at generating believable districts over time.

Read carefully, that sentence contains three constraints that are usually addressed by three incompatible mechanisms.

**It must learn.** Behavior at hour 100 must differ from behavior at hour 1 in a way that reflects what happened in between. A frozen checkpoint with a chat transcript does not satisfy this; the transcript is discarded at the context boundary and nothing survives it.

**It must learn from mistakes specifically.** Not from everything that happened — from the subset that carried a lesson. This is a *selection* problem before it is a storage problem, and selection implies a criterion that can be stated, audited, and disagreed with.

**It must stay accountable.** A world whose agents change behavior for reasons nobody can reconstruct is not a design surface; it is a bug that has not been noticed yet. Whatever mechanism carries the learning must also carry the provenance of each thing learned.

Continual fine-tuning fails all three in the same way. It is offline, so the learning is not runtime learning. It averages the lesson into  $10^{10}$  parameters, so the mistake and its correction are no longer separable objects. And it leaves no audit trail: the only honest answer to “why did the dockhand start refusing that job” is “the weights moved.”

Retrieval-augmented generation fixes provenance and fixes the runtime requirement, but as ordinarily built it fails a fourth constraint that a real-time world imposes and a chat product does not: **the retrieval must be cheap enough to run inside the decode loop, per agent, at frame rate, for a population.** An HNSW query per token per agent is not that. A vector database round trip per agent turn is not that either.

The AAMT stack has, in pieces, already solved each part of this. WP-20 established that the TERA cube admits a strict coarse-to-fine voxel hierarchy and honestly measured what that hierarchy can and cannot do alone (recall@10 of 0.017 on pure TERA routing — coarse routing is coarse). WP-21 inverted the index onto the output stream and proposed steering generation by geometry rather than by token. WP-22 mapped which Odu cells a trajectory actually covers. WP-23 made the index a single mmap-able file with 397 ns routed queries, a lineage journal, and a CRDT layer for multiple writers.

What no paper in the series has done is put them in one loop and let the loop run inside a game engine, at frame rate, with a gate on what gets written back. That is this paper.

### 1.1 A note on the borrowed vocabulary

Several load-bearing terms here are borrowed from wisdom traditions. The series’ convention (WP-24 §2 and §8) is to say where each came from and how far the correspondence is meant to carry, because a reader who assumes the vocabulary is doing argumentative work will over-read the results, and a reader who assumes it is decoration will miss where the structure is exact.

**Meji** and **Odu** name the 16 orthants and 256 cells of the TERA cube. The names are drawn from Ifá divination’s own binary/tetragram structure. WP-24 §8 argues the correspondence is structural rather than ornamental: four binary splits give  $2^4 = 16$  orthants as arithmetic, not as a naming choice. Nothing in this paper rests on that argument — substitute “orthant” and “cell” throughout and every claim is unchanged.

**Ma’at** (Egyptian) and **evenly yoked** (2 Corinthians 6:14) name the admission predicate  $W_{\text{action}}/F_{\text{agent}} \leq 1.0$  of §5.3. They are used as *names for a stated inequality*, not as an appeal to the authority of either tradition, and this paper makes no claim that the inequality is what either tradition meant. §11 states separately that the criterion encodes a design position rather than a derived optimum, and §8.3 experiment 7 is what would show whether it does the work claimed for it.

**HeartScale** is the platform’s existing module name for the component that evaluates that predicate. It is an identifier, not a claim about hearts.

**Breath**, in *breath cycle* and *BreathScheduler*, is a scheduling term: a  $\phi$ -ratioed expansion/contraction duty cycle (WP-17). No physiological claim is intended or implied.

---

## 2. Two artifacts that turn out to be one

The composition begins with an accident.

The platform produces two kinds of `.aamt` file. One is the WP-23 substrate: a hexadeca-tree over the TERA cube with payload records at the leaves, packed by `aamt pack`, opened by `mmap`, routed in `0(depth)`. The other is a model container: quantized weight tensors concatenated with a JSON manifest and a 32-byte footer, baked by `aamt-model.ts`, streamed into GPU buffers.

They were built by different tools for different consumers and they were given the same file extension and the same four-byte magic. Section 4 treats that as the defect it is. But the collision is worth pausing on, because the reason it happened is not carelessness. It happened because both artifacts genuinely are *the same kind of thing*: a large immutable blob, addressed by position, opened without parsing, streamed to a GPU on demand.

Take that seriously and the architecture writes itself. If experience and competence are both position-addressed immutable blobs, and if the position in both cases is a TERA coordinate, then:

- the agent’s **hidden state** projects to a TERA coordinate;
- that coordinate **routes** into the experience store, retrieving what this world has learned near this point in meaning-space;
- that same coordinate **predicts which weight pages** the next few tokens will touch, because a checkpoint’s activation pattern is not uniform over the archetype space;
- and the agent’s **deed**, if admitted, is written back at that coordinate, where the next agent to arrive at the same point in meaning-space will find it.

One address space. Three uses. The remainder of this paper is the consequences.

---

## 3. Architecture: competence and experience

### 3.1 The frozen tier

The competence tier is an ordinary decoder-only transformer, quantized and frozen. Nothing about it is novel and that is deliberate: every claim in this paper must survive the model being swapped.

The reference checkpoints are `BeyondSight-Flow.aamt` (10,413,570,119 bytes, 853 tensors,  **$26.90 \times 10^9$  parameters**) and `BeyondSight-GaIaN.aamt` (3,227,193,585 bytes, 400 tensors,  **$8.19 \times 10^9$  parameters**). Both are read directly from the manifest rather than derived; §9.5 records why the derived figures this section originally carried were wrong.

Neither checkpoint is uniformly quantized, and that turns out to matter for every argument below:

|                   | Flow                   | GaIaN                 |
|-------------------|------------------------|-----------------------|
| Ternary           | 25,621,954,560 (95.3%) | 7,567,003,648 (92.4%) |
| INT4              | 1,271,398,400 (4.7%)   | 621,236,224 (7.6%)    |
| f32 (unquantized) | 2,645,504 (0.01%)      | 308,224 (0.004%)      |
| Effective density | 0.387 B/param          | 0.394 B/param         |

|                            | Flow                   | GaIaN                  |
|----------------------------|------------------------|------------------------|
| Vocabulary $\times$ hidden | 248,320 $\times$ 5,120 | 151,669 $\times$ 4,096 |

The mix is deliberate and legible: `lm_head` is INT4, the embedding table and every projection are ternary, and the layer norms are left in f32 — 2.6 million parameters out of 26.9 billion, held at full precision because quantization noise on a norm weight lands on every activation in its layer rather than on one dot product.

The two vocabulary sizes are not interchangeable. Flow uses the 248,320-entry tokenizer, GaIaN the 151,669-entry one, and encoding either with the other’s vocabulary yields valid ids in a valid range, a forward pass that raises nothing, and word salad. This is why the tokenizer carries its vocabulary size in its header and asserts against the loaded model before encoding a single token.

Three quantization schemes are supported by the kernels, and the distinction matters later:

| Kind    | Weights per u32 | Alphabet                           | Bits/weight        |
|---------|-----------------|------------------------------------|--------------------|
| INT4    | 8               | $q \in [-8, 7]$ , nibble = $q + 8$ | 4                  |
| Ternary | 16              | $\{-1, 0, +1\}$ , codes 3 / 0 / 1  | 2 (1.58 effective) |
| 1-bit   | 32              | $\{-1, +1\}$                       | 1                  |

Group-wise scales are stored as f16, two per u32 word. The ternary path is the interesting one for residency (Section 6): it is the only alphabet with a *zero* symbol, which means a ternary page is compressible in a way an INT4 page is not, and sparsity in the weight matrix becomes sparsity in the page table.

### 3.2 The experience tier

The experience tier is a WP-23 substrate. Its properties are established there and used here without re-deriving them:

- `aamt_open` is `mmap` plus header validation — measured at 0.021 ms on a 1,000,000-record file, and independent of record count.
- A routed query costs  $O(\text{depth})$  tree descent with no scan — measured at 397 ns/query, 2.52 M queries/s single-threaded, through the full C ABI.
- The pack is immutable. Writes go through the CRDT layer above it (hybrid-logical-clock LWW map), which converges without coordination across multiple writers.
- Every query can be journaled to a content-hash-keyed sidecar and replayed bit-for-bit.

For a world, “multiple writers” is not a distributed-systems nicety. It is the default: every agent in every district is a writer, and they are writing concurrently into the same geometry. The CRDT layer WP-23 built for a browser tab and a desktop process turns out to be exactly the primitive an agent population needs.

### 3.3 The address that joins them

The projection from a TERA vector  $v = (T, E, R, A) \in [0, 1]^4$  to the 16-cell Meji distribution is the product-Bernoulli form established in WP-17 and implemented identically in Python `harmonic_engine`, TypeScript `@aamt/vortex-runtime`, and C# `MaiaM.Systems.Vortex.VortexEngine`:

$$p(M_k) = \prod_{d \in \{T, E, R, A\}} \begin{cases} v_d & \text{if } \text{bit}_d(k) = 1 \\ 1 - v_d & \text{otherwise} \end{cases}$$

with the exact recovery identity  $\text{recoverTera}(\text{mejiDistribution}(v)) = v$  holding to machine epsilon ( $\approx 2.22 \times 10^{-16}$ ), verified in all three targets.

Four derived scalars govern everything downstream:

- **RI** (resonance index) — probability mass on the dominant Meji.
- **BC** (breath coherence) — mass on Hamming-distance-1 neighbours of the dominant cell as a fraction of all non-dominant mass. High BC means a coherent transition; low BC means the state is scattered.
- $\beta$  — the TERA product, banded into shell colors.
- **S\_odu** — Shannon entropy of the distribution in bits; the scatter index.

The step this paper adds is the one that makes the coordinate available inside the decode loop at all: **projecting the model’s own hidden state into TERA space**, rather than projecting only the prompt text as `heartscale.ts`’s `textToTera` does.

Let  $h_t \in \mathbb{R}^d$  be the post-final-norm hidden state at decode step  $t$ . We take a fixed random projection  $P \in \mathbb{R}^{4 \times d}$  with orthonormalized rows, frozen at bake time and stored in the model container as the sentinel tensor `__tera_proj__`, and define

$$v_t = \sigma \left( \frac{Ph_t}{\sqrt{d}} \right) \in [0, 1]^4.$$

Three properties recommend this over a learned head. It requires no training, so it cannot be the thing that makes the architecture unfalsifiable. It is frozen and shipped with the model, so the coordinate is stable across sessions and across agents — a prerequisite for two agents’ experiences ever landing in the same cell. And it costs one  $4 \times d$  GEMV per token, which is negligible beside the  $\approx 7d^2$  of a transformer layer.

**What this is not.** It is not a claim that the random projection is *semantically* good. WP-20 already measured what coarse TERA routing achieves alone ( $\text{recall@10} = 0.017$ ) and this paper does not contradict it. The coordinate is a router, not a retriever; discrimination comes from the residual index below it, exactly as WP-20 prescribed. Section 8 states the experiment that would falsify even the routing claim.

---

#### 4. The format ambiguity, and its resolution

Both `.aamt` families begin with the ASCII bytes `AAMT` as a little-endian u32 magic `0x544D4141`. The substrate pack writes version 1. A single-tensor weight blob also writes version 1 in its v1 form. There is no type discriminator.

The two are separable only by what follows:

```
substrate pack v1:  [ 'AAMT' | u32 version=1 | u64 total_file_bytes | ... ]
single tensor v1:  [ 'AAMT' | u32 version=1 | u32 count | u32 groupSize | u32 groupCount ]
```

```

single tensor v2:  [ 'AAMT' | u32 version=2 | u32 kind | u32 count | u32 groupSize | u32 groupCount ]
model container:  [ tensor blob 0 ] ... [ manifest JSON ] [ 32 B footer ]
                  footer: [ 'AMDLE' | u32 version | u32 tensor_count | u32 pad
                           | f64 manifest_offset | f64 manifest_length ]

```

The model container is unambiguous — the AMDL footer is exact — so it is tested first. The substrate is then identified by the heuristic that the u64 at offset 8 equals the file’s own length on disk, which a tensor blob would match only if it happened to contain exactly length weights *and* zero in the adjacent word.

The reference implementation (AamtFileProbe) reads 32 bytes of head and 32 of tail, never the body, and returns a typed classification. This is stated explicitly as a **workaround, not a design**. The correct fix is a type byte in the shared header, agreed between libaamt and aamt-model.ts; until that lands, the ambiguity is confined to one class with the heuristic’s limits written down at the point of use, rather than being rediscovered inline at each call site.

Why a whitepaper section for a file-header bug: because the failure it caused was not a crash. Section 9.2 records what actually happened when a service that wanted a 63 KB geometric index was handed a 10.4 GB weight container.

---

## 5. Execution model: retrieval inside the decode loop

### 5.1 The loop

Per decode step  $t$ , after the final norm and before the LM head:

1. Project  $h_t \rightarrow v_t$  (§3.3). One  $4 \times d$  GEMV.
2. Compute the Meji distribution and its derived scalars. Sixteen products of four factors; free.
3. **Gate the retrieval.** If  $BC(v_t) < \tau_{BC}$  or  $S_{odu}(v_t) > \tau_S$ , skip retrieval this step. A scattered state is not near anything in particular, and retrieving for it injects noise. In the reference configuration this skips the majority of steps — see §8.
4. If not skipped, route  $v_t$  through the substrate and take the leaf bucket.
5. Convert retrieved records to a logit bias and add it (§5.2).
6. Apply the HeartScale gate to the biased logits, then sample.

Steps 3 and 4 are what make this affordable. The naive version — retrieve every step for every agent — is the version that does not run at frame rate. The coherence gate is not an optimization bolted on afterwards; it is the reason the loop closes, and it falls directly out of a quantity (BC) the stack already computes for other reasons.

### 5.2 Logit steering

Each retrieved record carries a payload with a token-level summary: the record stores, alongside its TERA coordinate, a sparse vector of token ids that occurred in the admitted deed together with their observed frequencies. For a retrieved bucket  $\mathcal{B}$  with per-record geometric weight  $w_r$  (falling off with Fisher–Rao distance from  $v_t$ ), the bias applied to logit  $\ell_i$  is

$$\ell'_i = \ell_i + \lambda \cdot BC(v_t) \cdot \sum_{r \in \mathcal{B}} w_r \log(1 + f_{r,i})$$

where  $\lambda$  is `substrateSteeringWeight` and  $f_{r,i}$  is token  $i$ 's frequency in record  $r$ .

Two deliberate choices. The bias scales with **BC**, so a coherent state is steered more strongly than a scattered one — the same signal that gates retrieval also modulates its strength, which means there is no discontinuity at the gate threshold. And  $\lambda = 0$  recovers the frozen model exactly, which makes “does the substrate do anything” a measurable question rather than an architectural assumption. Every claim about steering in §8 is stated as a delta against  $\lambda = 0$  on the same seeds.

Fisher–Rao rather than Euclidean distance, because TERA components live in  $[0, 1]$  and the interesting states are near the edges, where Euclidean distance badly misrepresents distinguishability. The engine already uses the Fisher–Rao geodesic for archetype transitions for exactly this reason.

### 5.3 The admission gate

Not everything an agent experiences should enter the shared substrate. A world whose agents write back indiscriminately converges on its own worst behavior: the most frequent outcome dominates the retrieval bucket, gets steered toward, becomes more frequent, and the loop is closed in the wrong direction. This failure mode is not hypothetical — it is the generic behavior of any retrieve-and-write loop without admission control, and it is why most such systems are read-only in practice.

The gate is the platform’s existing Ma’at criterion. A deed is *evenly yoked* when its action weight does not exceed the acting agent’s frequency:

$$\frac{W_{\text{action}}}{F_{\text{agent}}} \leq 1.0$$

`HeartScaleValidator.IsEvenlyYoked()` already gates action *execution*. The contribution here is to gate **writeback** with the same predicate, which gives the architecture a property worth stating precisely:

An agent may attempt anything it is capable of, but only what it could sustain enters the world’s memory.

A deed that fails the gate still happens. It is still visible to the player, it still has consequences in the scene, and it is still journaled locally. It simply does not become part of what the next agent inherits. Mistakes teach the agent that made them; only *survivable* mistakes teach the world.

Three admission classes, in increasing order of what they cost to get wrong:

| Class          | Predicate                                                                           | Written to                                                      |
|----------------|-------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| Local District | always<br>evenly yoked                                                              | agent-local journal, not shared district substrate, CRDT-merged |
| World          | evenly yoked $\wedge$ $\tau$ -permanent<br>(observed across $\geq 4$ breath cycles) | world substrate, requires a pack                                |



The  $\tau$ -permanence requirement on world-scope writes exists because a lesson observed once is an anecdote. WP-17's  $\tau$ -permanence — sustained across four breath cycles, weighted by RI and BC — is already defined and already computed per agent; requiring it here costs nothing new.

## 5.4 Provenance and replay

Every admitted record carries: the deed id, the acting agent id, the TERA coordinate at admission, the HLC timestamp, and the Ma'at ratio it passed with. Every *retrieval* that biased a decode step is journaled to the WP-23 sidecar, keyed by the substrate's content hash.

Together these give the property that motivated the whole design: for any behavior an agent exhibits, the question “why” has a finite, mechanical answer — this record, admitted by that agent, at that coordinate, at that time, with that Ma'at ratio, retrieved at these decode steps with these weights. And because WP-23's journal replays bit-for-bit against the content hash, the answer is verifiable rather than merely recorded.

This is the falsifiable form of the phrase “the world learns.” Not that its behavior drifts, which is easy and uninteresting, but that every increment of drift is attributable and replayable.

---

## 6. World-state weight residency

### 6.1 The problem stated honestly

BeyondSight-Flow.aamt is 10.4 GB. It currently lives in Unity's StreamingAssets, which means it is copied verbatim into every player build and, until the change recorded in §9.4, was tracked as a plain object in Git.

The ordinary responses are to quantize harder, distill, or stream from a server. All three are available and none is interesting. The interesting question is why the *whole* model would ever need to be resident when the agent asking is a fishmonger in the dock district.

### 6.2 Residency as a function of world state

The claim is not that a transformer has modular experts to be swapped — it does not, and this paper makes no MoE claim. The claim is narrower and empirical:

Over a decode trace, the distribution of activation magnitude across weight pages is non-uniform, and the non-uniformity correlates with the TERA coordinate of the states in that trace.

If that holds, then a page-granular residency policy keyed on the district's occupied Meji orthants keeps a working set materially smaller than the parameter count, and the pages evicted are the ones this district was never going to touch.

Note carefully: **this is stated as a hypothesis with a measurement attached, not as a result.** §8.3 gives the experiment. If the correlation is absent, the residency scheme reduces to ordinary LRU paging and this section's contribution is the mechanism, not the saving. That would be a real negative result and the series has a tradition of reporting those (WP-20 §4.3, WP-23 §8.8).

### 6.3 Mechanism

Weight pages are laid out in Morton order over the (layer, orthant) pair, using the same Morton.Encode3 the voxel field uses, so that pages serving nearby orthants are contiguous on disk and a residency change is a sequential read rather than a scatter.

The page table is the WP-23 tree itself. A page entry of 0 means *not resident* — the same vacuum convention the sparse voxel field uses for unallocated bricks, where a  $1024^3$  virtual field costs 8.4 MB of page table and a sparse occupancy of 4,096 voxels costs 33.5 MB of brick pool.

Paging is metered by `BreathScheduler`, which already time-slices token generation against a frame budget with an expansion:contraction ratio of  $\phi$ . Page-in work is issued during the expansion phase and page-out during contraction, so residency changes land in the part of the frame that already expects to be doing work. A district transition triggers a prefetch of the target district's orthant set one breath cycle ahead of the camera reaching it.

### 6.4 Why this is the demonstration

A residency map is a  $16 \times 16$  Odu heatmap that changes as the player walks. It is the WP-22 coverage map, rendered live, over weight pages instead of over trajectories. That makes it the rare architecture diagram that is also a runtime artifact and also a debugging tool: a district whose heatmap is fully saturated is a district whose design is not exercising the archetype space, and a district whose heatmap is a single hot cell is one whose agents are all the same person.

---

## 7. One math, three targets

The parity discipline in this stack is that the same math runs in Python (`harmonic_engine`), TypeScript + WGSL (`@aamt/voxel-runtime`), and C# + HLSL (`MaiiaM.Harmonic`, `MaiiaM.Systems.Vortex`), and that the agreement is checked rather than assumed.

The existing checks are real: `HarmonicMathTests` pins the Unity math against constants captured from the Python — `voxel_address([0.8, 0.6, 0.7, 0.3])` → meji 14 / odu 233 / key 3817, Fisher–Rao distance 1.61695, breath integrity 0.511043 — and includes a GPU-dispatch dequantization check.

What did not exist, and is the substantive engineering contribution of the Unity side of this paper, is parity coverage of the *kernels*. WP-23's cross-platform identity check compares a WASM build against a native build of the same C++ source. Here the two sides are different source languages compiled for different APIs: WGSL against a WebGPU device, HLSL against Unity's compute pipeline. Nothing structural forces them to agree, and §9.1 records what happened when nothing checked.

The harness has three tiers:

1. **Boot self-test.** One small ternary GEMV dispatched at engine start and compared against the scalar CPU reference, with a tolerance ( $1 \times 10^{-3}$ ) set to catch a mis-bound kernel — which produces zeros or garbage — without false-positiving on fp32 reassociation between a 64-wide GPU tree reduction and a sequential CPU sum.
2. **EditMode fixtures.** Golden vectors for each quantization kind and each kernel, tighter tolerances, run in CI.

3. **Cross-target vectors.** The same fixtures emitted by the TS engine and checked into the repository, so the Unity side is pinned to the browser engine and not merely to itself.

The CPU reference is written to be readable rather than fast, and is deliberately not a fallback execution path. Its job is to be something the GPU can be wrong against, and to be simple enough that when the two disagree you believe the CPU.

---

## 8. What is measured

This section separates measured, derived, and pending. The series' convention is that pending quantities are named with their harness rather than estimated, and that convention is followed here.

### 8.1 Measured

Inherited from prior papers and re-used unmodified:

- Substrate cold open: **0.021 ms**, 1,000,000-record file, independent of  $N$  (WP-23 §6).
- Routed query: **397 ns**, **2.52 M queries/s** single-threaded through the full C ABI (WP-23 §6).
- Pure-TERA retrieval recall@10: **0.017** (WP-20 §4.2) — the result that constrains §3.3's claim to routing rather than retrieval.
- Meji round-trip identity to  $\approx 2.22 \times 10^{-16}$  across all three targets.

Measured directly for this paper by parsing the shipped artifacts' footers and manifests (32 bytes of footer, ~119 KB of manifest; neither checkpoint's weights were read):

- BeyondSight-Flow.aamt: 10,413,570,119 B, AMDL v1 footer, **853 tensors**, manifest at offset 10,413,450,928 length 119,159 — terminating exactly at `fileLength - 32`. **26,895,998,464 parameters**: 25,621,954,560 ternary, 1,271,398,400 INT4, 2,645,504 f32. Effective density **0.3872 B/parameter**. Leading tensor `lm_head.weight` [248,320 × 5,120], .aamt blob header v2, kind INT4, 1,271,398,400 weights, group size 32, 39,731,200 groups — the header count equals the manifest shape product exactly, and the declared blob size equals the byte length the manifest records for that tensor exactly: 794,624,024 B = 635,699,200 B of packed nibbles + 158,924,800 B of group scales + a 24 B v2 header. (That is the tensor's own declared length, not the 119,159 B manifest length above — the two are distinct quantities.)
- BeyondSight-GaIaN.aamt: 3,227,193,585 B, **400 tensors**, **8,188,548,096 parameters** (7,567,003,648 ternary, 621,236,224 INT4, 308,224 f32), **0.3941 B/parameter**, vocabulary  $151,669 \times 4,096$ .
- **Flow is a hybrid checkpoint.** Its `__hybrid_meta__` sentinel declares 64 layers: **48 linear-attention and 16 full-attention**. Only the full-attention layers need a KV slot, so slot-indexed allocation is a measured **4.0×** reduction on this checkpoint, not an estimate.
- Two zero-length sentinel entries carry manifest-level metadata rather than tensor blobs: `__tera_meta__` (6,219 chars, per-layer) and `__hybrid_meta__` (770 chars).
- `oracle-demo.aamt`: 63,208 B substrate pack; mirrored variant 104,768 B.
- Reference WebGPU engine: **12,415 lines** TypeScript + **5,036 lines** WGS.
- Unity Vortex subsystem prior to this work: **1,300 lines** total, of which the GPU accelerator was **108**.

## 8.2 Derived

Nothing in this paper now rests on a derived parameter count. The figures above come from the manifests. §9.5 records the derivation that used to sit here, why it was wrong by 45%, and what it should have prompted.

## 8.3 Pending — the harness exists to produce these

| # | Quantity                                                                    | Harness                        | Falsifies                       |
|---|-----------------------------------------------------------------------------|--------------------------------|---------------------------------|
| 1 | Kernel parity: max abs. deviation, HLSL vs WGSL, per kind                   | MaiaM.Harmonic.Tests           | §7's one-math claim             |
| 2 | Decode throughput, tok/s, with $\lambda = 0$                                | boot self-test + logGpuProfile | nothing — baseline              |
| 3 | Retrieval gate skip rate at $\tau_{BC} = 0.5$                               | decode trace instrumentation   | §5.1's affordability claim      |
| 4 | Steering delta: task success at $\lambda > 0$ vs $\lambda = 0$ , same seeds | A/B decode harness             | §5.2 — the core claim           |
| 5 | Page-orthant mutual information over a decode trace                         | residency profiler             | §6.2 — the residency hypothesis |
| 6 | Working-set size per district vs full model                                 | residency telemetry            | §6's practical value            |
| 7 | Writeback contamination rate with and without the Ma'at gate                | population sim, $10^3$ deeds   | §5.3's admission argument       |

Experiment 4 is the one that matters. If steering produces no measurable improvement over the frozen model on the same seeds, the experience tier is decoration and this paper's central claim fails. It is stated first and plainly so that it cannot be quietly dropped.

Experiment 7 is the one most likely to produce an uncomfortable result, because the failure mode it tests for — convergence on the world's own worst behavior — may take longer than  $10^3$  deeds to appear.

## 8.4 Claims not made

- No claim that TERA routing retrieves well. It routes; WP-20 measured the rest.
- No MoE claim. §6 is a residency argument about activation locality, not a claim of modular experts.
- No claim that the random TERA projection is semantically optimal. It is frozen, cheap, and stable; whether a learned projection beats it is experiment 4's neighbour and is not attempted here.
- No throughput claim against llama.cpp, MLC, or any other engine. The kernels have not been benchmarked against an external baseline and saying so is cheaper than being caught.

- No claim that the Ma'at gate is the *correct* admission criterion. It is a stated, auditable criterion with a measurable contamination rate, which is a strictly weaker and strictly more defensible claim.

---

## 9. Corrections ledger: auditing the first implementation

In the EXP-01 tradition of recording what measurement falsified. Here the falsifying instrument is a code audit rather than a benchmark, and each entry records the *structural* change rather than the patch, because the patch is not the interesting part.

**9.1 The GPU accelerator was inert, and reported healthy.** VortexGpuEngine resolved its three compute kernels via `Resources.Load<ComputeShader>("Shaders/Compute/AamtMatmul")` while the `.compute` assets lived at `Assets/MaiaM/Shaders/Compute/` — outside any `Resources/` folder. Every load returned null; every dispatch took the `if (shader == null) return false` path; nothing logged. The scene reported an initialized accelerator and a registered system.

*Structural change:* the assets moved under `Assets/MaiaM/Resources/`, and resolution is no longer allowed to fail quietly — a failed load sets an explicit `EngineStatus.ShadersMissing` and logs the expected asset path. Silent-false returns were replaced throughout with a logged rejection reason.

**9.1a The uniforms could not have bound even if the shaders had loaded.** The HLSL declared its parameters as a struct inside `cbuffer ParamsBuffer : register(b0)`. Unity's `ComputeShader.SetInt` writes the implicit `$Globals` constant buffer, which an explicitly registered `cbuffer` is not part of. `SetInt("params_M", ...)` therefore wrote a slot no kernel read, and every dispatch would have executed with  $M = K = G = 0$  — writing nothing, raising nothing.

*Structural change:* the parameters are declared as flat top-level uniforms; the C# side holds them in a single `Uniform` name table with a comment on both sides stating that the two must move together. This is the failure the boot self-test (§7, tier 1) exists to catch, because it is invisible to every other form of testing: the code runs, returns success, and produces zeros.

**9.1b A latent constraint was undocumented and unenforced.** The ternary path reads one `uint4` (64 weights) per iteration and fetches exactly two scales for it. That is the correct group assignment only at group size 32. At 16 the second half-word silently reuses the first group's scale; at 64 both halves belong to one group but the second scale index runs past it. Neither errors.

*Structural change:* rejected at tensor construction with the derivation in the error message. The general lesson — that a kernel's unrolling factor can encode a hidden constraint on a parameter it never validates — is why every shape precondition is now checked at the boundary rather than assumed.

**9.2 The substrate service was pointed at a model container.** `AamtSubstrateService` defaulted to `aamt/BeyondSight-Flow.aamt` and called `AamtSubstrate.Open()` on it — handing a 10.4 GB weight container to a hexadeca-tree reader. The open failed, the failure was caught and logged at *warning* severity, and the service silently fell back to the 63 KB `oracle-demo.aamt` demo pack. Every downstream consumer then routed correctly against demo data while the scene setup reported the production model configured.

*Structural change:* §4's typed probe runs before any open; a model container produces an error naming what the file is and what the field wants, and the service does not fall back from a *miscon-*

*figuration* — only from a genuine unavailability of a correctly-typed file. Silent fallback across a category error is the specific pattern being removed, not the specific default.

**9.3 The replacement of the previous inference backend removed it without replacing it.** The scene-setup step named “AAMT Vortex AI Stack” destroyed the [Annika LLM] GameObject and logged that the legacy Gemma/LLMUnity model was “no longer needed.” But AAMTChatBridge, InnerThoughtsAgent and LocalGemmaProvider each independently reached into LLMUnity.LLMCharacter through System.Reflection, and the AAMT stack that replaced it contained a retrieval index and a GEMM primitive — no tokenizer, no KV cache, no attention, no sampler, no decode loop. After the step ran, every chat channel and every agent found no backend and went silent, with no error, because each reflection site degraded gracefully to “chat disabled.”

*Structural change:* a single IVortexInferenceSession seam. Three independent reflection sites became one interface with one implementation boundary, so a backend swap is one substitution rather than three coordinated edits, and a missing backend is one loud failure rather than three quiet degradations. The previous backend is retired *behind* the seam only once something satisfies it — the ordering that the original change inverted.

**9.4 Thirteen gigabytes were tracked as plain Git objects.** Both BeyondSight containers were committed without LFS and without a .gitattributes, making the repository effectively unclonable and baking 13.6 GB into every player build via StreamingAssets.

*Structural change:* LFS filters added, with the explicit note that this is *not* retroactive and that git lfs migrate import —everything rewrites history and must be coordinated. The build-size half of the problem is not solved by LFS at all and is deferred to the residency work in §6, where it is the motivating case rather than a packaging chore.

**9.5 This paper’s own derived parameter counts were wrong by 45%.** The first draft of §3.1 read the leading tensor’s header, saw INT4 group-32, and extrapolated the whole container at 0.5625 B/parameter — arriving at  $1.85 \times 10^{10}$  parameters for Flow and  $5.7 \times 10^9$  for GaIaN. Parsing the manifest gives  $26.90 \times 10^9$  and  $8.19 \times 10^9$ .

The error was assuming the container is uniformly quantized because its first tensor is. It is not: 95.3% of Flow is ternary at 2 bits, and reading `lm_head` — one of exactly two INT4 tensors in the file — as representative inflated the per-parameter cost by nearly half. Two smaller arithmetic slips rode along (0x4BC80000 read as 1,271,201,792 rather than 1,271,398,400, and 0x025E4000 as 39,600,128 rather than 39,731,200), both of which a single consistency check against the manifest shape product would have caught immediately — and did, the moment the container loader ran against the real file.

Three things follow, and the third is the one worth keeping:

1. The measured density is **0.387 B/parameter**, better than the derived figure claimed, because the checkpoint is more aggressively quantized than assumed — the error was not flattering.
2. §6’s residency argument is *strengthened*. It observed that ternary is the only alphabet with a zero symbol and therefore the only one whose sparsity becomes page-table sparsity. On the assumption of an INT4-dominant container that was a marginal note; at 95.3% ternary it applies to almost the entire model.
3. **A derived figure was quoted where a measured one was available.** The manifest was 119 KB at a known offset and answered the question exactly. The discipline this series states — separate measured from derived, name the harness — is not only about not fabricating

numbers; it is about noticing when a quantity did not need deriving at all. §8.2 is now empty on purpose.

**9.6 What the audit did not find.** Worth stating, because a ledger that only records failures misrepresents the artifact. The pure mathematics port (VortexEngine, MejiCatalog, MaiaM.Harmonic) is faithful, parity-tested against both other targets, and required no correction. The kernels themselves — once bound — implement the WGSF semantics correctly, including the awkward parts: the f16 scale unpacking, the three quantization alphabets, the groupshared tree reduction. VortexService’s use of the Fisher–Rao geodesic rather than a linear interpolation for archetype transitions is metrically correct near the cube edges where TERA states actually live, and is the kind of detail that is usually got wrong. The defects were in the seams, not in the mathematics — which is the ordinary shape of this failure and worth naming as such.

---

## 10. Positioning

Against **continual fine-tuning**: this architecture does not update weights and does not claim to. It claims that a large class of what fine-tuning is used for in agent deployments — carrying forward situational lessons — is better served by an append-only store with provenance, because the lesson stays a separable, inspectable, revocable object.

Against **ordinary RAG**: the difference is the gate and the geometry. RAG retrieves by embedding similarity, admits by ingestion policy, and runs per turn. This runs per token with a coherence gate that skips most steps, admits by a stated behavioral criterion, and shares one address space with the residency mechanism.

Against **MoE and modular architectures**: no overlap. Those change what the model *is*; this changes what is resident and what is retrieved. The two compose.

Against **memory-augmented agent frameworks**: the distinguishing property is that the memory is the same artifact type as the weights, opened the same way, addressed in the same coordinates. That is what makes §6 possible at all, and it is the thing that would be lost by swapping the substrate for a vector database with a nicer API.

---

## 11. Limitations and honest framing

**The central claim is unmeasured.** §8.3 experiment 4 has not been run. Every architectural argument in §5 stands on the hypothesis that geometric steering improves behavior over the frozen model, and until that A/B exists this paper describes a mechanism, not a result. This is stated plainly rather than softened.

**The TERA projection is arbitrary.** A frozen random projection is defensible as a *stable* coordinate, not as a good one. It may place semantically distant states in the same cell often enough to make retrieval actively harmful, which would show up as experiment 4 producing a *negative* delta.

**The Ma’at gate encodes a value judgment.** “Only what an agent could sustain enters shared memory” is a design position, not a derived optimum. It will systematically exclude high-cost lessons — precisely the dramatic ones — and whether that produces a wiser world or a blander one is an empirical question this paper does not answer.

**Multi-writer convergence is verified for two writers, not for a population.** WP-23’s CRDT property testing used a desktop process and a browser tab. A district of forty agents writing concurrently is a different regime, and the failure modes of LWW under high-contention identical-coordinate writes — where every agent in a crowd projects to the same cell — are not characterized.

**Residency assumes activation locality that has not been demonstrated.** §6.2 is explicit that this may reduce to LRU paging.

**Nothing here has run on a shipping build.** The corrections in §9 were made against a project that does not currently compile-and-run in a measured configuration on target hardware. The engineering is real; the claim that it works is pending §8.3.

---

## 12. Reproducibility

| Component                    | Location                                             |
|------------------------------|------------------------------------------------------|
| Vortex mathematics (C#)      | Assets/MaiaM/Systems/Vortex/VortexEngine.cs          |
| GPU kernels (HLSL)           | Assets/MaiaM/Resources/Shaders/Compute/*.compute     |
| GPU dispatch + self-test     | Assets/MaiaM/Systems/Vortex/VortexGpuEngine.cs       |
| CPU reference                | Assets/MaiaM/Systems/Vortex/VortexCpuReference.cs    |
| Inference seam               | Assets/MaiaM/Systems/Vortex/IVortexInferenceSession. |
| Format probe                 | Assets/MaiaM/Systems/Substrate/AamtFileProbe.cs      |
| Substrate service            | Assets/MaiaM/Systems/Substrate/AamtSubstrateService. |
| Harmonic math + parity tests | Assets/MaiaM/Harmonic/                               |
| WebGPU reference engine      | packages/voxel-runtime/                              |
| Vortex CPU reference (TS)    | packages/vortex-runtime/                             |
| Substrate core               | packages/libaamt/                                    |

Parity fixtures run from Unity’s Test Runner (EditMode → Run All). The boot self-test runs automatically and logs its maximum absolute deviation. Seeds are fixed (1729 for the boot test).

## 13. Conclusion

A world whose agents learn from their mistakes needs three things that are usually traded against each other: runtime adaptation, per-lesson provenance, and a cost that fits inside a frame. The AAMT stack had already built each piece for a different reason — a geometric index for memory, a steering mechanism for long-form generation, a coherence measure for archetype transitions, an ethical predicate for action execution. Composing them yields an architecture where the model stays frozen and auditable, the world’s accumulated experience is an append-only file with a lineage journal, and the criterion for what the world remembers is stated rather than emergent.

The same coordinate that addresses experience also predicts which weights the next tokens will touch, which turns a model too large to ship into one whose working set is bounded by where the player is standing rather than by its parameter count.

Whether the composition actually improves behavior is experiment 4, and it has not been run. What can be said now is narrower and still worth saying: the mechanism is specified, the seams



that made the first implementation silently inert have been closed and documented, and the measurement that would falsify the central claim is written down before the result is known.

---

### **Related work in this series**

- **WP-15** — Vortex-Addressed Semantic Memory: retrieval by TERA coordinate.
- **WP-17** — Product-Bernoulli Hypercube Family: the projection this paper uses.
- **WP-20** — Voxel-Addressable Memory: the coarse-to-fine hierarchy, and the recall measurement that bounds §3.3.
- **WP-21** — Voxel-Steered Autoregressive Generation: the steering primitive §5.2 instantiates.
- **WP-22** — Odu Coverage Maps: the visualization §6.4 reuses over weight pages.
- **WP-23** — The .aamt Substrate: the experience tier’s storage, routing, journal, and CRDT layer.
- **WP-25** — Substrate Integration Guide: the binding surface this paper’s Unity target consumes.
- **EXP-01** — Topologically Protected Trit Memory: the corrections-ledger tradition §9 follows.